

Bucket Sort

O Bucket Sort distribui os elementos em vários "baldes" (buckets) de acordo com seus valores. Depois, cada balde é ordenado individualmente e os seus elementos são reunidos no vetor original.

Funcionamento

1. Criar uma quantidade de baldes, normalmente relacionada ao tamanho do vetor.
2. Distribuir cada elemento no balde correspondente.
3. Ordenar cada balde separadamente.
4. Concatenar os baldes em ordem.

bucket	radix msd
<pre>void bucketSort(float v[], int n) { if (n <= 0) return; float** baldes = new float*[n]; int* tamanhos = new int[n]; int capacidade = n; for (int i = 0; i < n; i++) { baldes[i] = new float[capacidade]; tamanhos[i] = 0; } for (int i = 0; i < n; i++) { int indice = (int)(v[i] * n); if (indice == n) indice = n - 1; baldes[indice][tamanhos[indice]++] = v[i]; } for (int i = 0; i < n; i++) { for (int j = 1; j < tamanhos[i]; j++) { float atual = baldes[i][j]; int k = j - 1; while (k >= 0 && baldes[i][k] > atual) { baldes[i][k + 1] = baldes[i][k]; k--; } baldes[i][k + 1] = atual; } } int posicao = 0; for (int i = 0; i < n; i++) { for (int j = 0; j < tamanhos[i]; j++) v[posicao++] = baldes[i][j]; delete[] baldes[i]; } delete[] baldes; delete[] tamanhos; }</pre>	<pre>void radixSortMSD(int v[], int n) { if (n <= 1) return; int maior = v[0]; for (int i = 1; i < n; i++) { if (v[i] > maior) maior = v[i]; } int divisor = 1; while (maior / divisor >= 10) divisor *= 10; int* auxiliar = new int[n]; auto ordenar = [&](auto&& ordenar, int inicio, int fim, int div) -> void { if (inicio >= fim div == 0) return; int contagem[10] = {0}; for (int i = inicio; i <= fim; i++) { int digito = (v[i] / div) % 10; contagem[digito]++; } int inicioGrupo[10]; inicioGrupo[0] = inicio; for (int digito = 1; digito < 10; digito++) inicioGrupo[digito] = inicioGrupo[digito - 1] + contagem[digito - 1]; int proximaPosicao[10]; for (int digito = 0; digito < 10; digito++) proximaPosicao[digito] = inicioGrupo[digito]; for (int i = inicio; i <= fim; i++) { int digito = (v[i] / div) % 10; auxiliar[proximaPosicao[digito]++] = v[i]; } for (int i = inicio; i <= fim; i++) v[i] = auxiliar[i]; for (int digito = 0; digito < 10; digito++) { int grupoInicio = inicioGrupo[digito]; int grupoFim = grupoInicio + contagem[digito] - 1; ordenar(ordenar, grupoInicio, grupoFim, div / 10); } }; ordenar(ordenar, 0, n - 1, divisor); delete[] auxiliar; }</pre>

Radix MSD

O Radix Sort MSD(Most Significant Digit) ordena inteiros começando pelo dígito mais significativo. Os elementos são separados em grupos de acordo com esse dígito; depois, cada grupo é processado recursivamente usando o próximo dígito.

Funcionamento

1. Encontrar o maior valor para descobrir o maior divisor decimal.
2. Separar os elementos pelo dígito mais significativo.
3. Repetir a separação recursivamente para cada grupo, usando o próximo dígito.
4. Parar quando não houver mais dígitos para analisar

A complexidade de algoritmos mede como o tempo de execução (complexidade temporal) ou o uso de memória (complexidade espacial) de um programa cresce à medida que o tamanho da entrada de dados (n) aumenta. Em vez de medir o tempo em segundos — o que varia de acordo com o processador ou a linguagem —, medimos a complexidade pelo número de operações fundamentais que o algoritmo executa. Organizadas da mais eficiente / mais rápida para a menos eficiente / mais lenta: $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Complexidades: Se n é a quantidade de elementos, d é a quantidade de dígitos e b é a base para o RADIX.

Caso	Bucket	Radix MSD
Melhor	$O(n)$	$O(d(n + b))$
Pior	$O(n^2)$	$O(d(n + b))$
Médio	$O(n)$	$O(d(n + b))$
Estável	Depende de como os baldes são ordenados	Não necessariamente
In-place	Não	Não
Memória	$O(n)$	$O(n + b)$
Método	Distribuição	Distribuição por dígitos

Notação	Nome	O que representa?	Analogia Prática
O	Big O	Limite Superior (Worst Case)	"Na pior das hipóteses, leva no máximo este tempo."
Ω	Big Ômega	Limite Inferior (Best Case)	"Na melhor das hipóteses, leva pelo menos este tempo."
Θ	Big Teta	Limite Justo (Tight Bound)	"O algoritmo cresce exatamente nesta taxa (piso = teto)."

$O(1)$ — Tempo Constante: O que significa: A resposta vem de forma direta e imediata. O tempo não muda, independente se $n=1$ ou $n=1.000.000$. Como funciona na prática: O algoritmo não percorre nada; ele calcula a posição exata de memória via fórmula ou chave direta. Exemplos de Algoritmos/Operações: Acessar um índice direto em um vetor (vetor[500]). Buscar por chave em uma Tabela Hash (std::unordered_map). **$O(\log n)$ — Tempo Logarítmico:** O que significa: O espaço de busca é cortado ao meio a cada passo. Como funciona na prática: A cada comparação, metade das opções restantes é descartada sem nem precisar ser lida. Exemplos de Algoritmos/Operações: Busca Binária em vetores ordenados. Busca e inserção em Árvores Binárias de Busca balanceadas (AVL, Red-Black). **$O(\sqrt{n})$ — Raiz Quadrada:** O que significa: Crescimento intermediário; infinitamente mais rápido que $O(n)$, porém mais lento que $O(\log n)$. Como funciona na prática: O laço de repetição avança testando a multiplicação do contador por ele mesmo ($i * i \leq n$). Exemplos de Algoritmos/Operações: Teste de Primalidade de um número. Decomposição em Raiz Quadrada (SQRT Decomposition). **$O(n)$ — Tempo Linear:** O que significa: O trabalho cresce na mesma proporção exata que o tamanho da entrada n . Como funciona na prática: O algoritmo é obrigado a realizar uma varredura/passada completa por todos os n elementos da estrutura. Exemplos de Algoritmos/Operações: Busca Linear em vetor desordenado. Somar ou encontrar a média de uma lista. Percorrer todos os nós de uma Árvore/Grafo (DFS / BFS). **$O(n \log n)$ — Tempo Linearítmico:** O que significa: O problema é sucessivamente dividido ao meio ($\log n$ níveis), e em cada nível de divisão é feita uma varredura completa por todos os n elementos. Como funciona na prática: Ao contrário da busca binária que descarta metades, aqui todas as metades são processadas e reconstruídas. Exemplos de Algoritmos/Operações: Algoritmos eficientes de ordenação por comparação: Merge Sort, Quick Sort (caso médio), Heap Sort. nível de divisão $\times$ varredura por nível = $\log_2 n \times n$. **$O(n^2)$ — Tempo Quadrático:** O que significa: Para cada um dos n elementos, o algoritmo percorre novamente todos os n elementos. Como funciona na prática: Ocorre tipicamente com dois laços aninhados. Exemplos de Algoritmos/Operações: Algoritmos simples de ordenação: Bubble Sort, Selection Sort, Insertion Sort. Percorrer todas as combinações de pares em um vetor. Varredura em matrizes quadradas de dimensões $N \times N$. **$O(2^n)$ e $O(n!)$ — Tempos Exponencial e Fatorial:** O que significa: Crescimento catastrófico. O tempo dobra ou multiplica exponencialmente a cada novo elemento inserido. Como funciona na prática: Tenta resolver o problema por força bruta, gerando todas as combinações ou permutações possíveis. Exemplos de Algoritmos/Operações: Fibonacci recursivo ingênuo ($O(2^n)$). Gerar todos os subconjuntos de um conjunto ($O(2^n)$). Problema do Caxeiro-Viajante por força bruta ($O(n!)$). Gerar todas as permutações de uma string ($O(n!)$).

n (Elementos)	$O(1)$	$O(\log n)$	$O(\sqrt{n})$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
8	1	3	2	8	24	64	256
64	1	6	8	64	384	4.096	$\$1,8 \times 10^{19}$
1.024	1	10	32	1.024	10.240	1.048.576	Impraticável
1.000.000	1	20	1.000	1.000.000	20.000.000	10^{12}	Impraticável